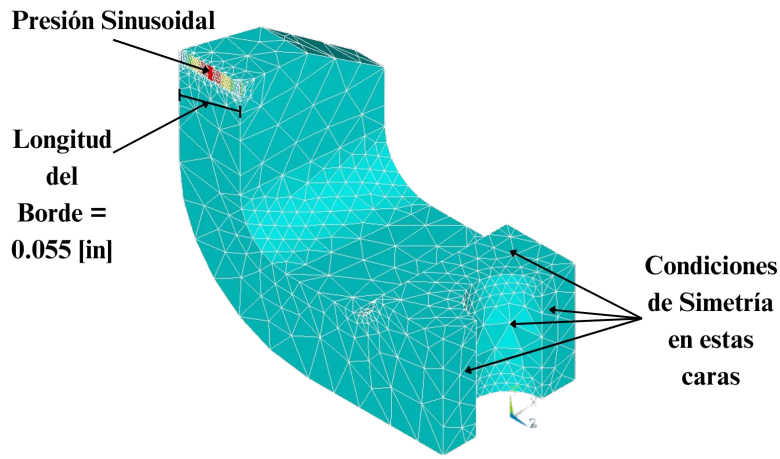
# 1 Análisis Estructural de un Cortador de Torno




## 1.1 Objetivo

En este notebook, se realizará un análisis estructural detallado de un cortador de torno utilizando ANSYS Mechanical APDL (MAPDL). Este tipo de análisis es crucial para prever el comportamiento del cortador bajo condiciones de carga reales y asegurar su rendimiento y seguridad en operaciones de manufactura.

## 1.2 Contenido

- Modelar geométricamente un cortador de torno en ANSYS MAPDL.
- Aplicar condiciones de carga y simular el comportamiento bajo estas cargas.
- Analizar las tensiones y deformaciones resultantes para evaluar la integridad estructural del cortador.
- Visualizar los resultados para una interpretación clara y detallada de las tensiones y deformaciones.

ANSYS MAPDL será utilizado para: - Importar y preparar la geometría del cortador. - Definir materiales, propiedades, y tipos de elementos. - Aplicar condiciones de frontera y cargas. - Resolver el modelo de elementos finitos. - Post-procesar los resultados para obtener visualizaciones de las tensiones y deformaciones principales.

## 1.3 Paso 1: Configuración Inicial

[ ]:
```
# Importamos las librerías necesarias
import os
import numpy as np
from ansys.mapdl.core import launch_mapdl
from ansys.mapdl.core.examples.downloads import download_example_data

PI = np.pi
EXX = 1.0e7
NU = 0.27
Fuerza = 10000
```

En este paso inicial, se importan las bibliotecas necesarias y se lanza la sesión de ANSYS MAPDL en modo SMP para aprovechar múltiples núcleos del procesador.

```
[ ]: # Configuramos el directorio actual y algunas constantes básicas
path = os.getcwd()  # Obtiene el directorio de trabajo actual
PI = np.pi  # Constante de Pi para cálculos trigonométricos

# Inicializar ANSYS MAPDL
mapdl = launch_mapdl(additional_switches="-smp")  # Lanza MAPDL en modo SMP

# Reiniciar la base de datos de MAPDL y configurar el entorno
mapdl.clear()  # Limpia cualquier configuración anterior en MAPDL
```

### 1.4 Paso 2: Importación y Preparación de la Geometría

```
[ ]: # Cargar la geometría del modelo y establecer parámetros
archivo_geometria = download_example_data("LatheCutter.anf", "geometry")
mapdl.input(archivo_geometria)  # Carga la geometría desde un archivo
mapdl.finish()  # Termina el procesamiento del input
longitud_presion = mapdl.parameters["PRESS_LENGTH"]  # Extrae un parámetro de␣
 ↪MAPDL
```

Cambias unidades y titulo

```
[ ]: mapdl.units("Bin")
mapdl.title("Cortador de Torno")
```

Propiedades del material

```
[ ]: mapdl.prep7()  # Prepara el procesador de pre-procesamiento
mapdl.mp("EX", 1, EXX)  # Define el módulo de elasticidad
mapdl.mp("NUXY", 1, NU)  # Define el coeficiente de Poisson
```

El tipo de elemento MAPDL `SOLID285` se utiliza con fines de demostración. Considere utilizar un tipo de elemento o densidad de malla apropiados para su aplicación real.

```
[ ]: mapdl.et(1, 285)  # Establece el tipo de elemento a usar
mapdl.smrtsize(4)  # Tamaño inteligente de los elementos
mapdl.aesize(14, 0.0025)  # Tamaño específico para la malla
mapdl.vmesh(1)  # Malla el volumen seleccionado

# Definir condiciones de simetría y cargar la configuración de presión
mapdl.da(11, "symm")  # Aplica condiciones de simetría
# Similar para otros lados o caras del modelo
mapdl.da(16, "symm")
mapdl.da(9, "symm")
mapdl.da(10, "symm")
```

## 2 Paso 3: Sistema de coordenadas y carga

Cree un Sistema de Coordenadas local (CS) para la presión aplicada como una función de X local.

El ID del CS local es 11

[ ]:
```
# Configurar y aplicar la carga de presión
mapdl.cskp(11, 0, 2, 1, 13)  # Crea un sistema de coordenadas local
mapdl.csys(1)  # Cambia al sistema de coordenadas creado
mapdl.view(1, -1, 1, 1)  # Configura la vista en MAPDL
mapdl.psymb("CS", 1)  # Establece símbolos para la vista
mapdl.vplot(
    color_areas=True,
    show_lines=True,
    cpos=[-1, 1, 1],
    smooth_shading=True,
)
```

Los trazados VTK no muestran símbolos de trazado MAPDL. Sin embargo, para utilizar las puede establecer la opción de palabra clave `vtk` en `False`.

[ ]:
```
mapdl.lplot(vtk=False)
```

## 3 Paso 4: Carga de presión

Cree una carga de presión, cárguela en MAPDL como un array de tablas, verifique la y resuelve.

[ ]:
```
# Define y aplica una carga variable a lo largo de una dimensión
puntos = 10  # Número de puntos para definir la carga
longitud_x = np.linspace(0, longitud_presion, puntos)  # Define la distribución␣
↪de la carga
presion = Fuerza * np.sin(PI * longitud_x / longitud_presion)  # Calcula la␣
↪carga como función del seno
presion = np.stack((longitud_x, presion), axis=-1)  # Combina x con la presión
mapdl.load_table("MY_PRESS", presion, "X", csysid=11)  # Carga la tabla de␣
↪presiones en MAPDL
```

`length_x` y `press` son vectores. Para combinarlos en la forma necesaria para definir el array de la tabla MAPDL, puedes utilizar numpy.stack.

[ ]:
```
# Selecciona áreas y aplica la carga de presión
mapdl.asel("S", "Area", "", 14)
mapdl.nsla("S", 1)
mapdl.sf("All", "Press", "%MY_PRESS%")
mapdl.allsel()  # Selecciona todo de nuevo
```

Configurar la solución.

[ ]:
```
# Configura y resuelve el modelo
mapdl.finish()  # Finaliza la configuración
mapdl.slashsolu()  # Accede al solucionador
mapdl.nlgeom("On")  # Activa la geometría no lineal si es necesario
mapdl.psf("PRES", "NORM", 3, 0, 1)  # Configura opciones de presión
mapdl.solve()  # Resuelve el análisis
mapdl.finish()  # Finaliza el solucionador
```

# 4 Paso 5: Trazado

[ ]:
```
mapdl.post1()
mapdl.set("last")
mapdl.allsel()

mapdl.post_processing.plot_nodal_principal_stress("1", smooth_shading=False)
```

# 5 Trazado - Parte del modelo

[ ]:
```
mapdl.csys(1)
mapdl.nsel("S", "LOC", "Z", -0.5, -0.141)
mapdl.esln()
mapdl.nsle()
mapdl.post_processing.plot_nodal_principal_stress(
    "1", edge_color="white", show_edges=True
)
```

# 6 Trazado - Opciones de leyenda

[ ]:
```
mapdl.allsel()
sbar_kwargs = {
    "color": "black",
    "title": "1st Principal Stress (psi)",
    "vertical": False,
    "n_labels": 6,
}
mapdl.post_processing.plot_nodal_principal_stress(
    "1",
    cpos="xy",
    background="white",
    edge_color="black",
    show_edges=True,
    scalar_bar_args=sbar_kwargs,
    n_colors=9,
)
```

Vamos a probar algunas opciones de barra escalar de la PyVista documentación. Por ejemplo, pongamos texto negro sobre fondo beige. beige.

Las palabras clave de la barra escalar definidas como un diccionario Python son un método alternativo alternativo al uso de {clave:valor}. Puede utilizar el método de hacer clic y arrastrar para reposicionar la barra escalar. Haga clic con el botón izquierdo del ratón y manténgalo pulsado mientras mueve el ratón. mientras mueve el ratón.

[ ]:
```
sbar_kwargs = dict(
    title_font_size=20,
    label_font_size=16,
    shadow=True,
    n_labels=9,
    italic=True,
    bold=True,
    fmt="%.1f",
    font_family="arial",
    title="1st Principal Stress (psi)",
    color="black",
)

mapdl.post_processing.plot_nodal_principal_stress(
    "1",
    cpos="xy",
    edge_color="black",
    background="beige",
    show_edges=True,
    scalar_bar_args=sbar_kwargs,
    n_colors=256,
    cmap="jet",
)
```

# 7 Paso 6: Tratamiento posterior

## 7.1 Lista de resultados

Obtenga todas las tensiones nodales principales.

[ ]:
```
mapdl.post_processing.nodal_principal_stress("1")
```

Obtiene las tensiones principales nodales del subconjunto de nodos.

[ ]:
```
mapdl.nsel("S", vmin=1200, vmax=1210)
mapdl.esln()
mapdl.nsle()

print("«Los números de los nodos son:»")
print(mapdl.mesh.nnum) # obtener los números de los nodos
```

```
print("«Las tensiones nodales principales son:»")
mapdl.post_processing.nodal_principal_stress('1')
```

[ ]:
```
mapdl.exit()
```